

Java 객체지향프로그래밍을 활용한 GitHub 프로젝트 품질 점검 시스템 구현

- 객체지향 프로그래밍 과제 보고서 -

학 과 : 인공지능소프트웨어학과

학 년 : 1학년

학 번 : 2601110288

성 명 : 박 수 용

1. 주제

본 프로젝트의 주제는 "GitHub 프로젝트 품질 점검 시스템"입니다. 사용자가 자신의 GitHub 프로젝트 정보를 입력하면, 프로젝트 유형에 맞는 기준으로 품질 점수와 등급을 계산하고, 부족한 부분에 대한 개선 조언을 출력해 주는 Java 콘솔 프로그램입니다.

평가 항목은 README 작성 여부, 배포 여부, 커밋 수, 기술스택 개수, 프로젝트 설명 길이의 다섯 가지이며, 100점 만점을 기준으로 점수를 산출한 뒤 A+부터 D까지의 등급을 부여합니다.

2. 주제 선택 배경

대학생이나 개발 입문자는 GitHub에 프로젝트를 올리더라도 README가 부족하거나, 커밋 수가 적거나, 기술스택 설명이 없거나, 배포 링크가 없는 경우가 많습니다. 저 또한 포트폴리오를 정리하면서 "내 프로젝트가 남들이 보기에 얼마나 완성도 있게 보일까" 하는 점이 늘 고민이었습니다.

그래서 프로젝트의 완성도를 객관적인 점수로 점검해 볼 수 있다면 유용하겠다고 생각하였고, 이를 Java 객체지향 프로그래밍으로 구현해 보기로 하였습니다. 특히 프로젝트 유형(웹, AI, Java)마다 중요하게 평가되는 항목이 다르다는 점을 "상속과 오버라이딩"으로 자연스럽게 표현할 수 있다고 판단하여 이 주제를 선택하였습니다.

3. 객체지향 개념

3.1 적용한 객체지향 개념

본 프로젝트에는 다음과 같이 객체지향의 주요 개념을 적용하였습니다.

개념	적용 위치	설명
클래스	전체 8개 클래스	역할별로 클래스를 분리하여 구현하였습니다.
추상화	Project (abstract)	공통 틀을 추상 클래스로 정의하고 구체 구현은 자식에게 맡겼습니다.
상속	Web/AI/JavaProject	Project 클래스를 상속받아 공통 코드를 재사용하였습니다.
생성자	Project, 자식 클래스	생성자로 필드를 초기화하고 자식은 super()로 호출하였습니다.
오버라이딩	calculateQualityScore()	유형별로 점수 계산 방식을 다르게 재정의하였습니다.
접근제한자	필드 private / 메서드 public	외부 접근을 제한하여 데이터를 보호하였습니다.
다형성	ProjectManager	부모 타입(Project) 변수로 자식 객체를 처리하였습니다.
캡슐화	Project 필드 + getter	private 필드를 getter를 통해서만 접근하도록 하였습니다.

3.2 프로젝트 유형별 차등 점수 설계 (핵심)

이 프로그램의 핵심은 프로젝트 유형마다 항목별 가중치를 다르게 둔 점입니다. 모든 유형의 합계는 100점으로 동일하지만, 웹 프로젝트는 "배포"를, AI 프로젝트는 "기술스택과 설명"을, Java 프로젝트는 "커밋 수와 설명"을 더 중요하게 평가합니다.

평가 항목	WebProject	AIProject	JavaProject
README 여부	20	20	20
배포 여부	30	10	15
커밋 수	15	20	25
기술스택 개수	20	25	15
설명 길이	15	25	25
합계	100	100	100

이렇게 가중치를 다르게 둬으로써, 동일한 calculateQualityScore() 메서드를 호출하더라도 객체의 실제 유형에 따라 서로 다른 결과가 나오게 됩니다. 즉, 상속·오버라이딩·다형성이 형식적으로만 들어간 것이 아니라 실제 기능으로 동작하도록 설계하였습니다.

4. 주요 소스 코드 및 설명

전체 코드 중 객체지향 개념이 잘 드러나는 부분만 발췌하여 설명하겠습니다.

4.1 object 클래스 — 추상화, 캡슐화, 생성자

```
public abstract class Project { // 추상 클래스

    // 필드는 전부 private (캡슐화)
    private String projectName;
    private int commitCount;
    private boolean hasReadme;
    // ... (그 외 필드 생략)

    // 생성자 - 객체 생성 시 필드 초기화
    public Project(String projectName, int commitCount,
                   boolean hasReadme) {
        this.projectName = projectName;
        this.commitCount = commitCount;
        this.hasReadme = hasReadme;
    }

    // getter - private 필드 읽기용
    public int getCommitCount() { return commitCount; }

    // 추상 메서드 - 자식이 반드시 구현해야 함
    public abstract int calculateQualityScore();
}
```

Project는 모든 프로젝트의 공통 정보를 담는 부모 추상 클래스입니다. 필드를 전부 private으로 선언하여 외부에서 직접 접근하지 못하게 하고(캡슐화), getter를 통해서만 값을 읽도록 하였습니다. 또한 calculateQualityScore()를 추상 메서드로 선언하여 자식 클래스가 반드시 구현하도록 강제하였습니다(추상화).

4.2 WebProject 클래스 — 상속, 오버라이딩, super

```
public class WebProject extends Project {    // Project 상속

    // 생성자에서 부모 생성자(super)를 호출
    public WebProject(/* 매개변수 생략 */) {
        super(/* 인자 생략 */);
    }

    // 오버라이딩 - 웹은 배포(30점)에 가중치를 높게 둬
    @Override
    public int calculateQualityScore() {
        double score = 0;
        score += isHasReadme() ? 20 : 0;
        score += isDeployed() ? 30 : 0;           // 배포 30점
        score += getCommitRate() * 15;
        score += getTechStackRate() * 20;
        score += getDescriptionRate() * 15;
        return (int) Math.round(score);
    }
}
```

WebProject는 extends 키워드로 Project를 상속받습니다(상속). 생성자에서는 super()를 호출하여 부모의 생성자로 공통 필드를 초기화합니다. 그리고 @Override를 사용해 calculateQualityScore()를 웹 프로젝트에 맞게 재정의하였습니다(오버라이딩). 배포 여부에 30점을 부여하여 웹 프로젝트의 특성을 반영하였습니다. AIProject와 JavaProject도 같은 방식으로 각자의 가중치에 맞게 이 메서드를 오버라이딩합니다.

4.3 ProjectManager 클래스 — 다형성

```
// 부모 타입 변수 하나로 어떤 자식 객체든 담을 수 있음 (다형성)
Project p;

if (type == 1) {
    p = new WebProject(/* ... */);
} else if (type == 2) {
    p = new AIProject(/* ... */);
} else {
    p = new JavaProject(/* ... */);
}

// p의 실제 타입에 따라 다른 calculateQualityScore()가 실행됨
int score = p.calculateQualityScore();
```

사용자가 선택한 유형에 따라 서로 다른 자식 객체를 생성하지만, 모두 부모 타입인 Project 변수 p 하나로 받아 처리합니다(다형성). 이후 p.calculateQualityScore()를 호출하면, p에 실제로 담긴 객체의 종류에 따라 알맞은 오버라이딩 메서드가 자동으로 실행됩니다. 덕분에 유형이 늘어나도 호출 코드는 그대로 유지할 수 있습니다.

5. 프로그램 실행 화면 (UI / 입력 / 출력)

5.1 실행 메뉴 화면

프로그램을 실행하면 아래와 같이 메인 메뉴가 표시됩니다.

```
=====
      GitHub 프로젝트 품질 점검 시스템
=====

1. 프로젝트 품질 점검 시작
2. 프로그램 설명 보기
3. 종료
메뉴 선택 >>
```

5.2 프로그램 설명 보기

이 프로그램은 GitHub 프로젝트 품질을 점검하는 시스템입니다.

[평가 항목]

- README 작성 여부
- 배포 여부
- 커밋 수
- 기술스택 개수
- 설명 글자 수

[등급 기준]

- 90점 이상 → A+
- 80점 이상 → A
- 70점 이상 → B
- 60점 이상 → C
- 60점 미만 → D

※ 프로젝트 유형마다 항목별 가중치가 다릅니다.

5.3 실행 예시 A — 완성도 높은 웹 프로젝트 (높은 점수)

- 입력값

```
-- 프로젝트 유형 선택 --
1. Web Project
2. AI Project
3. Java Project
선택 >> 1
프로젝트 이름 : MyPortfolio
사용 언어 : Java
GitHub 주소 : https://github.com/psy0635-ctrl/Java\_Project
커밋 수 : 45
README 있나요? (y/n) : y
배포했나요? (y/n) : y
기술스택 개수 : 6
설명 글자 수 : 150
```

- 출력 결과

```
=====
                프로젝트 품질 점검 결과
=====
프로젝트명   : MyPortfolio
유형         : 웹 프로젝트
사용 언어    : Java
GitHub 주소  : https://github.com/psy0635-ctrl/Java\_Project
-----
커밋 수      : 45회
README      : 있음
배포 여부   : 완료
기술스택     : 6개
설명 길이   : 150자
-----
품질 점수    : 100 / 100점
등급         : A+ (매우 우수)
평가        : 포트폴리오에 바로 사용할 수 있는 우수한 프로젝트입니다!
-----
[개선 조언]
- 모든 항목이 잘 갖춰져 있습니다!
=====
```

5.4 실행 예시 B — 보완이 필요한 Java 프로젝트 (낮은 점수)

- 입력값

```
-- 프로젝트 유형 선택 --
1. Web Project
2. AI Project
3. Java Project
선택 >> 3
프로젝트 이름 : MyPortfolio
사용 언어 : Java
GitHub 주소 : https://github.com/psy0635-ctrl/Java\_Project
커밋 수 : 3
README 있나요? (y/n) : n
배포했나요? (y/n) : n
기술스택 개수 : 1
설명 글자 수 : 10
```

- 출력 결과

```
=====
                프로젝트 품질 점검 결과
=====
프로젝트명   : MyPortfolio
유형         : Java 프로젝트
사용 언어    : Java
GitHub 주소  : https://github.com/psy0635-ctrl/Java\_Project
-----
커밋 수      : 3회
README      : 없음
배포 여부   : 미배포
기술스택     : 1개
설명 길이   : 10자
-----
품질 점수    : 20 / 100점
등급         : D (포트폴리오 보완 필요)
평가        : 포트폴리오로 사용하기엔 부족합니다. 적극적인 보완이 필요합니다.
-----
[개선 조언]
- README 파일을 추가하세요.
- 프로젝트를 배포해보세요.
- 커밋이 너무 적습니다. 최소 10회 이상 권장합니다.
- 기술스택을 좀 더 다양하게 써보세요.
- 프로젝트 설명이 너무 짧습니다.
=====
```

5.5 동일 입력에 대한 유형별 점수 비교

같은 프로젝트 정보(README 있음, 미배포, 커밋 12회, 기술스택 4개, 설명 60자)를 유형만 다르게 하여 점검하면, 아래와 같이 점수와 등급이 달라집니다. 이는 오버라이딩과 다형성이 실제로 동작함을 보여 줍니다.

항목 (동일 입력)	WebProject	AIProject	JavaProject
README 있음	20	20	20
배포 미완료	0	0	0
커밋 12회 (75%)	11.25	15	18.75
기술스택 4개 (75%)	15	18.75	11.25
설명 60자 (75%)	11.25	18.75	18.75
합계 (반올림)	58	73	69
등급	D	B	C

배포를 중요하게 평가하는 웹 프로젝트는 배포가 없어 점수가 가장 낮고, 기술스택과 설명을 중시하는 AI 프로젝트는 상대적으로 높은 점수를 받았습니다.

6. GitHub 주소

본 프로젝트의 전체 소스 코드는 아래 GitHub 저장소(public)에서 확인할 수 있습니다.

```
https://github.com/psy0635-ctrl/Java_Project/tree/main/src/Java_Project2
```

7. 결론 및 느낀 점

이번 과제를 통해 클래스, 생성자, 상속, 오버라이딩, 접근제한자, 다형성, 캡슐화, 추상화 등 객체지향의 핵심 개념을 하나의 프로그램 안에서 직접 적용해 볼 수 있었습니다. 특히 프로젝트 유형별로 점수 계산 방식을 다르게 두면서, 상속과 오버라이딩이 단순한 형식이 아니라 실제로 의미 있게 동작하도록 설계한 점이 가장 기억에 남습니다.

처음에는 모든 유형이 같은 방식으로 점수를 계산하도록 만들었는데, 그렇게 하면 자식 클래스를 나눈 의미가 없다는 것을 깨닫고 유형별 가중치를 다르게 설계하였습니다. 이 과정에서 “왜 객체지향을 사용하는가”에 대해 조금 더 이해하게 되었습니다. 앞으로 더 많은 유형을 추가하더라도 기존 코드를 거의 수정하지 않고 확장할 수 있다는 점에서 객체지향의 장점을 체감할 수 있었습니다.